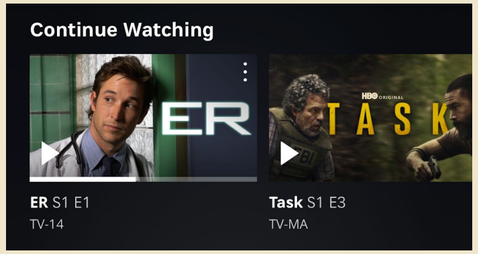
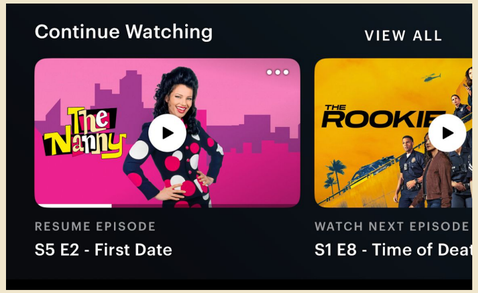
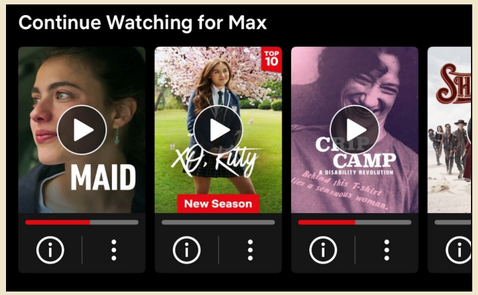
The Problem



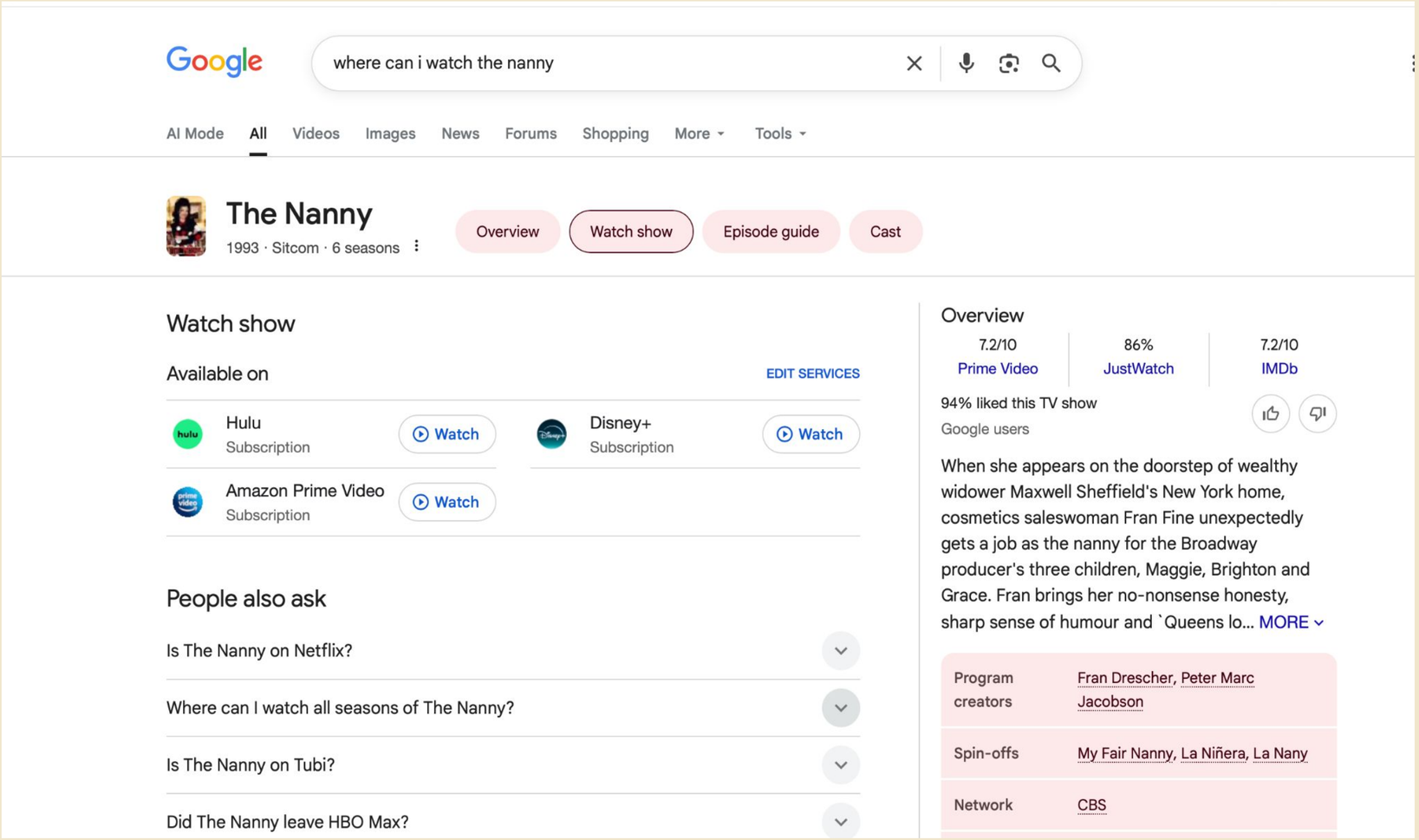
HBO MAX



HULU

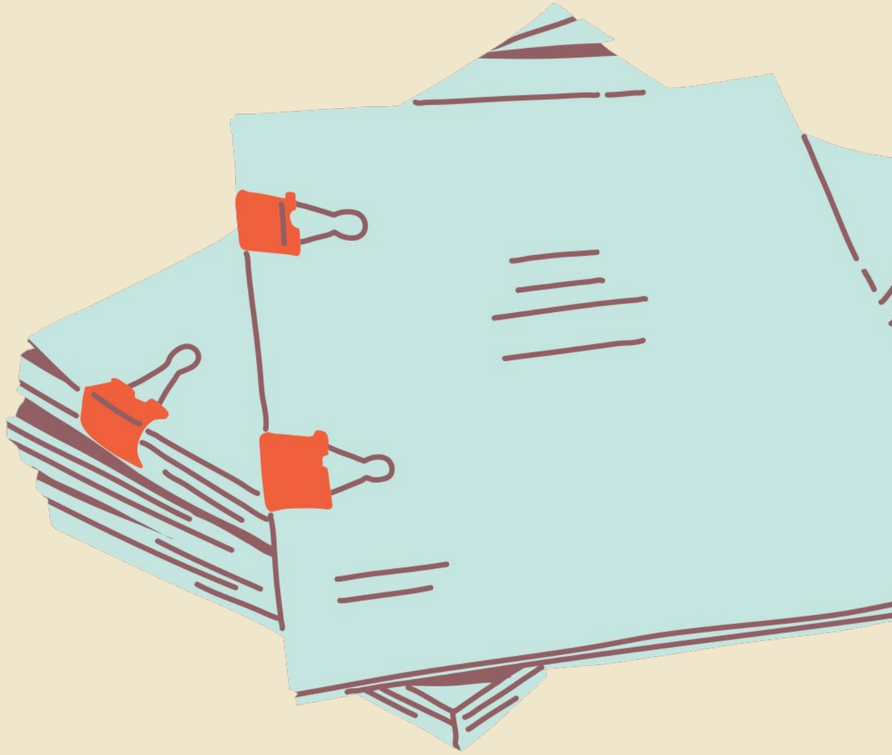


NETFLIX



[pillar*boxd*]

Presentation Flow



01
Tools We Used
& Methodologies

02
Architecture Diagram

03
Challenges

04
Demo

OUR TEAM

MEL

API INTEGRATION ENGINEER

- Wired the frontend-backend connection
- Built API route handlers

MAX

FULL-STACK LEAD

- Architected the core backend and server skeleton
- Established the technical foundation the rest of the team built on
- Set up the CI/CD pipeline

CAM

DATABASE ENGINEER

- Designed and implemented the database schema
- Built the login and registration

ALEX

FRONTEND ENGINEER

- Built the UI and visual experience of the app
- Built home, search, watchlist, and recommendations pages

AHMED

NOTIFICATIONS ENGINEER

- Built the Discord webhook notifications tied into CI/CD
- Auto-alerted the team when builds or tests failed on any branch
- Built Notifications system that lets user be notified when a new episode for a TV Show they're following drops

KORDELL

PROJECT MANAGER / SCRIBE

- Maintained project docs, meeting minutes, and release notes
- Managed the GitHub Project board and tracked user stories
- Ensured weekly TA deliverables were submitted on time

01

TOOLS WE USED

OUR TOOLKIT

GITHUB PROJECTS 5/5

PROJECT TRACKING

Kanban boards that live inside GitHub. Cards move across Backlog, In Progress, and Done, with PRs and commits linked directly to each task.

FIGMA 5/5

DESIGN & WIREFRAMES

We wireframed the home page, navbar, and show cards before writing code, so the team agreed on the UI visually before building.

HTML / CSS 5/5

UI

The building blocks of every page: HTML defines structure, CSS controls how it looks.

NODE.JS 5/5

APPLICATION SERVER

The runtime that lets JavaScript outside of a browser. Both our Hono backend and Next.js frontend rely on it under the hood.

JEST 5/5

TESTING

JavaScript testing framework used for our backend routes. Works natively with TypeScript via ts-jest, matching our stack.

POSTGRES 5/5

DATABASE

Relational database storing users, shows, and watchlists. Hosted on Supabase, which adds auth and APIs on top of PostgreSQL.

RENDER 5/5

DEPLOYMENT

Cloud hosting that pulls from GitHub and deploys automatically. Beginner-friendly and free for our project size.

FROM CLASS STACK TO OUR STACK

CLASS TOOL	TOOL WE USED	WHY WE SWITCHED	EXAMPLE IN OUR CODE
Express	**Hono**	Hono is TypeScript-native, lighter, and more modern. Same concept, both are server frameworks, but Hono has better TypeScript support	In **server/routes/auth.ts,** Hono gives us **c.req.json()** which is fully typed. Express would require separate bodyParser middleware and return any typed bodies. Our entire server has zero “any” types because of this.
Handlebars	**Next.js + React**	Handlebars renders HTML on the server with templates. Next.js + React is a more modern approach that gives you a proper interactive frontend with options to optimize rendering.	Search results update instantly without a full page reload. With Handlebars the server re-renders the entire HTML page on every search. With Next.js + React only the results section updates, giving users a faster experience when browsing shows.
Pg.promise	**Supabase**	Supabase gives you a database, authentication, and row-level security all in one.	In **server/routes/auth.ts,** user registration is one line: **supabase.auth.signUp({ email, password }).** With pg-promise we would have had to manually write password hashing with bcrypt, session management, and token generation ourselves like the lab code required.
Mocha/Chai	**Jest**	Both are test frameworks that do the same job. Jest is more widely used in the TypeScript/React ecosystem and has better TypeScript support via ts-jest. Mocha and Chai also don’t do as well with mocking API responses.	in **server/jest.config.ts,** ts-jest compiles TypeScript test files on the fly with no separate build step. With Mocha we would have needed to compile TypeScript to JavaScript first before running any tests. Our test files like **auth.test.ts** import TypeScript files directly.

Methodologies

Agile + Iterative

Weekly TA meetings with incremental feature development each week.

Peer Code Reviews

PRs required at least one teammate approval before merging to main.

Branching

Every task had its own branch. We never worked directly on main.

CI/CD

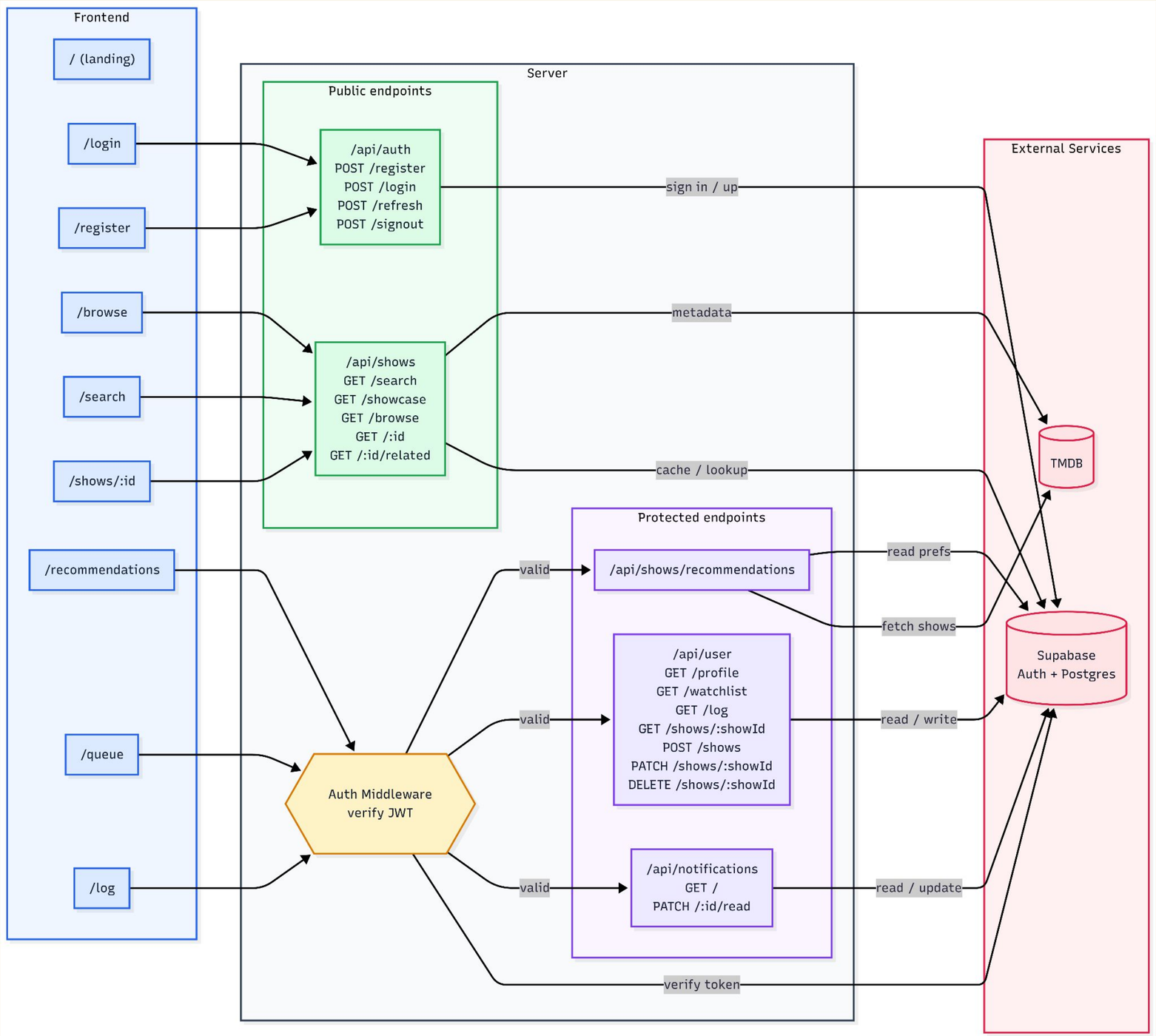
GitHub Actions ran tests automatically on every push and alerted us in Discord on failure.

Pair Programming

Coordinated between frontend and backend outside of whole-team meetings when needed.

02

ARCHITECTURE DIAGRAM


Frontend
/ (landing)
/login
/register
/browse
/search
/shows/:id
/recommendations
/queue
/log
Server
Public endpoints
/api/auth
POST /register
POST /login
POST /refresh
POST /signout
/api/shows
GET /search
GET /showcase
GET /browse
GET /:id
GET /:id/related
sign in / up
metadata
cache / lookup
Protected endpoints
/api/shows/recommendations
/api/user
GET /profile
GET /watchlist
GET /log
GET /shows/:showId
POST /shows
PATCH /shows/:showId
DELETE /shows/:showId
/api/notifications
GET /
PATCH /:id/read
Auth Middleware
verify JWT
valid
valid
valid
verify token
read prefs
fetch shows
read / write
read / update
External Services
TMDB
Supabase
Auth + Postgres

03

CHALLENGES

WHY GROUP CHALLENGES COMPOUNDED

CAUSE

Large group size

EFFECT

More moving parts meant any underlying issue was amplified across the team.

CAUSE

Diverse skill sets + new tools

EFFECT

Work clustered around the most confident members instead of being distributed evenly.

CAUSE

Early communication issues

EFFECT

Task assignment became harder, creating redundancy and lost time.

04

Demo

Future Scope & Enhancements

Social Features

TODAY

Everything is private to the user, watchlists and activity aren't shared.

WHAT'S NEXT

Let users follow friends, see what they're watching, and share watchlists.

A friends system could also power taste-based recommendations from people who watch similar shows.

Smarter Recommendations

TODAY

Recommendations use vector embeddings to find shows similar to what you've already watched.

WHAT'S NEXT

Factor in ratings, completion rate, streaming services, time-of-day, and collaborative filtering ("users like you also loved X").

This would make recommendations significantly more personalized.

Group Parties

THE IDEA

Let multiple users build a shared watchlist together, useful for couples or friend groups deciding what to watch.

HOW IT WORKS

Users vote on shows and the app surfaces the highest-voted pick.

Builds directly on the existing watchlist and user systems.